K.R. Mangalam University

School of Engineering &
Technology Operating
System Lab (ENCA252)
Lab Assignment 2
Banker's Algorithm -Deadlock Avoidance

Submitted by:

Name: kshitij Marwah

Roll Number: 2401201021

Course: BCA (AI & DS)

Submitted To:

Dr. Jyoti Yadav

Objective

To implement the Banker's Algorithm for Deadlock Avoidance and determine whether the system is in a safe state. The safe sequence of process execution is identified and the system state is analyzed.

1. Tools & Technologies Used

Tool / Technology	Purpose
Python 3.x	Used to implement Banker's Algorithm
Built-in Libraries	Used for input handling and matrix operations
Terminal (Linux/Windows)	Used to execute the program and view output

2. Key Concepts

Term	Formula	Description
Allocation Matrix	---	Resources currently held by each process
Maximum Matrix	---	Maximum resources each process may need
Need Matrix	Max - Allocation	Remaining resources each process still needs
Available	---	Resources currently free in the system
Safe Sequence	---	Order in which all processes can finish without deadlock

3. Task 1 — System Input & Data Representation

Objective:

To understand how resources are allocated to processes in an operating system.
The user inputs the number of processes and resources, followed by the Allocation Matrix, Maximum Matrix, and Available Resources.

Input Data Used:

Number of Processes: 5 Number of Resources: 3

Allocation Matrix

Process	R0	R1	R2
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

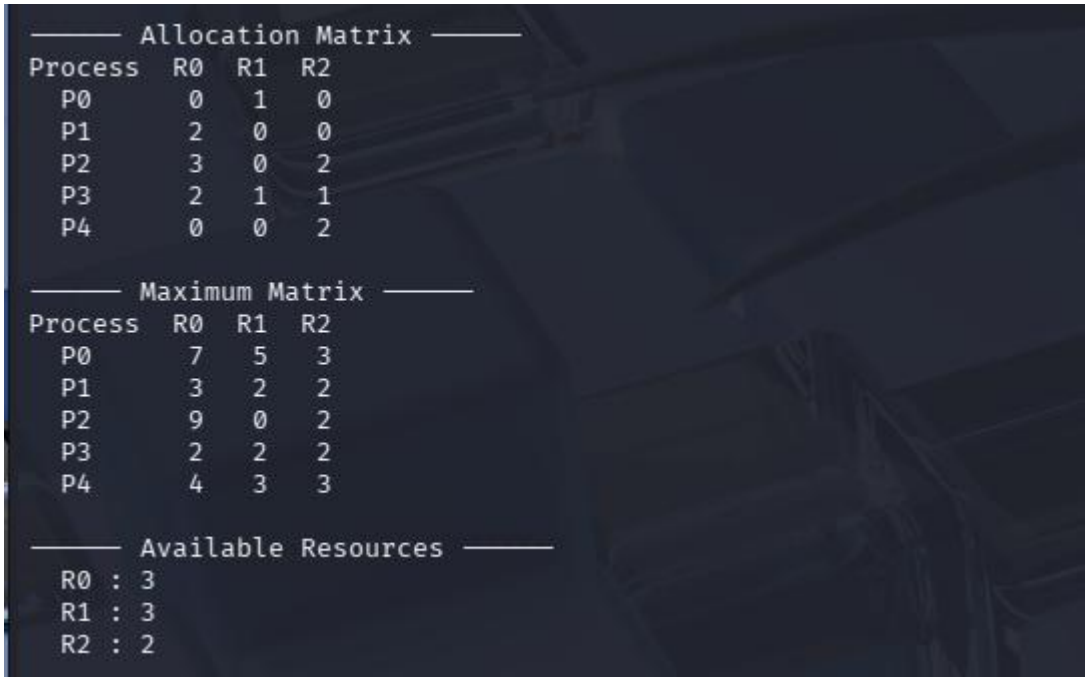
Maximum Matrix

Process	R0	R1	R2
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2
P4	4	3	3

Available Resources

R0	R1	R2
3	3	2

Screenshot — Task 1 Output



```
────── Allocation Matrix ──────
Process  R0  R1  R2
  P0      0   1   0
  P1      2   0   0
  P2      3   0   2
  P3      2   1   1
  P4      0   0   2

────── Maximum Matrix ──────
Process  R0  R1  R2
  P0      7   5   3
  P1      3   2   2
  P2      9   0   2
  P3      2   2   2
  P4      4   3   3

────── Available Resources ──────
  R0 : 3
  R1 : 3
  R2 : 2
```

4. Task 2 — Need Matrix Calculation

Objective:

To compute the remaining resource requirements for each process.
Need Matrix is calculated using the formula: Need = Maximum – Allocation

Need Matrix

Process	R0	R1	R2
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1

Screenshot — Task 2 Output

```
────── Need Matrix (Max - Allocation) ──────
Process  R0  R1  R2
  P0      7   4   3
  P1      1   2   2
  P2      6   0   0
  P3      0   1   1
  P4      4   3   1
```

5. Task 3 — Banker's Safety Algorithm

Objective:

To determine whether the system is in a safe state using the Banker's Safety Algorithm.

Steps Followed:

- Initialize Work = Available = [3, 3, 2]
- Initialize Finish array as False for all processes
- Find a process whose Need <= Work, run it, add its Allocation to Work
- Mark that process as Finished
- Repeat until all processes finish or no process can proceed

Screenshot — Task 3 Output

```
────── Safety Algorithm Result ──────
  System is in SAFE STATE
```

6. Task 4 — Safe Sequence Determination

Objective:

To find and display the safe execution order of all processes.

Safe Sequence:

P1 --> P3 --> P4 --> P0 --> P2

The system is in a SAFE STATE. All 5 processes can complete execution without deadlock.

Screenshot — Task 4 Output

```
────── Safe Sequence ──────
  Safe Sequence is : P1 ⟶ P3 ⟶ P4 ⟶ P0 ⟶ P2
```

7. Task 5 — Result Analysis

Metric	Result
System State	SAFE STATE

Safe Sequence	P1 --> P3 --> P4 --> P0 --> P2
Deadlock Possibility	No — all processes can complete

Analysis

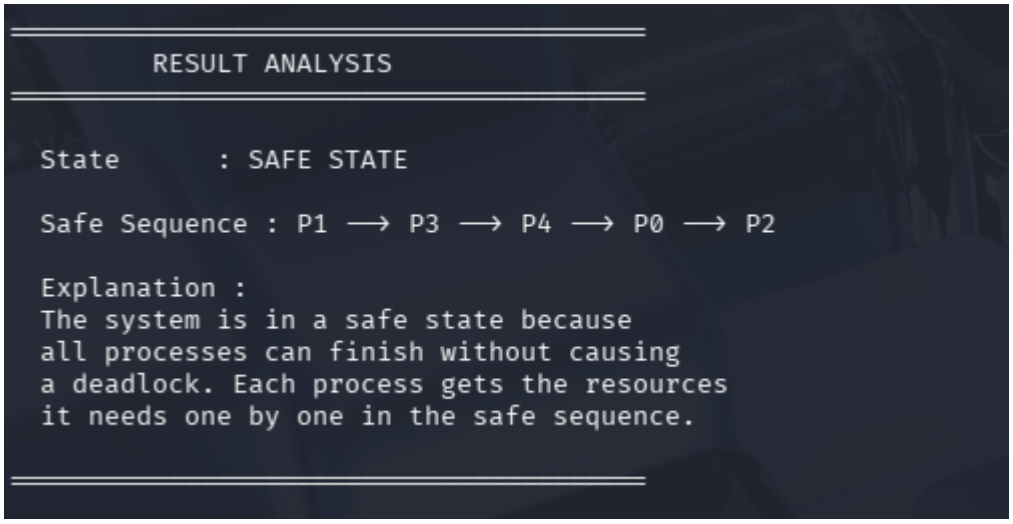
- The system is in a safe state because a valid safe sequence exists
- Each process gets the resources it needs in the correct order
- No process has to wait indefinitely for resources

Conclusion

Banker's Algorithm successfully avoided deadlock in this case because:

- A safe sequence was found: P1 → P3 → P4 → P0 → P2
- Available resources were sufficient to satisfy each process one by one
- However, Banker's Algorithm requires prior knowledge of maximum resource needs

Screenshot — Task 5 Output



```
═══════════════════════════════════════
        RESULT ANALYSIS
═══════════════════════════════════════

State      : SAFE STATE

Safe Sequence : P1 ⟶ P3 ⟶ P4 ⟶ P0 ⟶ P2

Explanation :
The system is in a safe state because
all processes can finish without causing
a deadlock. Each process gets the resources
it needs one by one in the safe sequence.

═══════════════════════════════════════
```

GitHub Repository

Link: [Github]

Source Code

The complete source code (bankers_algorithm.py) is attached below as screenshots.

Banker.py > ...

```python
    print("  P" + str(i) + "      ", end="")
    for j in range(r):
        print(str(need[i][j]) + "   ", end="")
    print()

# -------------------------------------------------------
# Task 3 - Banker's Safety Algorithm
# -------------------------------------------------------

work = available[:]
finish = [False] * n
safe_sequence = []
count = 0

while count < n:
    found = False
    for i in range(n):
        if finish[i] == False:
            can_run = True
            for j in range(r):
                if need[i][j] > work[j]:
                    can_run = False
                    break
            if can_run == True:
                for j in range(r):
                    work[j] = work[j] + allocation[i][j]
                finish[i] = True
                safe_sequence.append(i)
                found = True
                count = count + 1
    if found == False:
        break

all_done = True
for i in range(n):
    if finish[i] == False:
        all_done = False

print("\n------ Safety Algorithm Result ------")
if all_done == True:
    print("  System is in SAFE STATE")
else:
    print("  System is in UNSAFE STATE - Deadlock may occur")

# -------------------------------------------------------
```

Ln 122, Col 2 Spaces: 4 UTF-8 CRLF { } Python 3.14.2 Python 3.14 (64-bit)

Banker.py > ...

```python
print("Process  ", end="")
for j in range(r):
    print("R" + str(j) + "  ", end="")
print()
for i in range(n):
    print("  P" + str(i) + "      ", end="")
    for j in range(r):
        print(str(allocation[i][j]) + "   ", end="")
    print()

# display maximum matrix
print("\n------ Maximum Matrix ------")
print("Process  ", end="")
for j in range(r):
    print("R" + str(j) + "  ", end="")
print()
for i in range(n):
    print("  P" + str(i) + "      ", end="")
    for j in range(r):
        print(str(maximum[i][j]) + "   ", end="")
    print()

# display available
print("\n------ Available Resources ------")
for j in range(r):
    print("  R" + str(j) + " : " + str(available[j]))

# -------------------------------------------------------
# Task 2 - Need Matrix Calculation
# Need = Maximum - Allocation
# -------------------------------------------------------

need = []
for i in range(n):
    row = []
    for j in range(r):
        row.append(maximum[i][j] - allocation[i][j])
    need.append(row)

print("\n------ Need Matrix (Max - Allocation) ------")
print("Process  ", end="")
for j in range(r):
    print("R" + str(j) + "  ", end="")
print()
for i in range(n):
```

Ln 26, Col 46 Spaces: 4 UTF-8 CRLF { } Python 3.14.2 Python 3.14 (64-bit)

Banker.py > ...

```python
import copy

# -------------------------------------------------------
# Task 1 - System Input and Data Representation
# -------------------------------------------------------

print("=" * 50)
print("   Banker's Algorithm - Deadlock Avoidance")
print("=" * 50)

n = int(input("\nEnter number of processes : "))
r = int(input("Enter number of resources : "))

# allocation matrix
print("\nEnter Allocation Matrix (row by row):")
allocation = []
for i in range(n):
    row = []
    print("  Process", i, ":")
    for j in range(r):
        val = int(input("    Resource " + str(j) + " : "))
        row.append(val)
    allocation.append(row)

# maximum matrix
print("\nEnter Maximum Matrix (row by row):")
maximum = []
for i in range(n):
    row = []
    print("  Process", i, ":")
    for j in range(r):
        val = int(input("    Resource " + str(j) + " : "))
        row.append(val)
    maximum.append(row)

# available resources
print("\nEnter Available Resources:")
available = []
for j in range(r):
    val = int(input("  Resource " + str(j) + " : "))
    available.append(val)

# display allocation matrix
print("\n------ Allocation Matrix ------")
print("Process  ", end="")
```

Ln 26, Col 46 Spaces: 4 UTF-8 CRLF { } Python 3.14.2 Python 3.14 (64-bit)

Banker.py > ...

```python
# Task 4 - Safe Sequence Determination
# -------------------------------------------------------

print("\n------ Safe Sequence ------")
if len(safe_sequence) == n:
    print("  Safe Sequence is : ", end="")
    for i in range(len(safe_sequence)):
        print("P" + str(safe_sequence[i]), end="")
        if i != len(safe_sequence) - 1:
            print(" --> ", end="")
    print()
else:
    print("  No safe sequence found. System is UNSAFE.")

# -------------------------------------------------------
# Task 5 - Result Analysis
# -------------------------------------------------------

print("\n========================================")
print("          RESULT ANALYSIS")
print("========================================")

if all_done == True:
    print("\n  State      : SAFE STATE")
    print("\n  Safe Sequence : ", end="")
    for i in range(len(safe_sequence)):
        print("P" + str(safe_sequence[i]), end="")
        if i != len(safe_sequence) - 1:
            print(" --> ", end="")
    print()
    print("\n  Explanation :")
    print("  The system is in a safe state because")
    print("  all processes can finish without causing")
    print("  a deadlock. Each process gets the resources")
    print("  it needs one by one in the safe sequence.")
else:
    print("\n  State      : UNSAFE STATE")
    print("\n  Explanation :")
    print("  The system is in an unsafe state because")
    print("  no safe sequence of execution was found.")
    print("  Some processes are waiting for resources")
    print("  that cannot be fulfilled, leading to deadlock.")

print("\n========================================")
```

Ln 122, Col 2 Spaces: 4 UTF-8 CRLF { } Python 3.14.2 Python 3.14 (64-bit)